Students:
Nguyen Huu Truong Giang [u6110634@udg.edu]

Lab 5: Task-Priority kinematic control (2A)

1 Introduction

In robot arm control, the robot often needs to satisfy additional constraints, such as joint limits or obstacle avoidance, while attempting to accomplish a primary goal task. These constraints are commonly formulated as inequality tasks. To address this type of control problem, task-priority frameworks with inequality constraints are employed, allowing mandatory tasks to be strictly satisfied while still striving to achieve the desired task.

In this lab, we introduce two types of inequality tasks: joint limit avoidance and end-effector obstacle avoidance. These inequality tasks are combined with an end-effector position control task within a task-priority framework to evaluate the behavior of the robot arm under different constraint configurations and target end-effector positions.

Section 2 presents the task definitions for each inequality constraint. Section 3 illustrates the experimental results under different scenarios and discusses the key observations. Finally, Section 4 concludes the lab.

2 Methodology

The implementation was carried out in Python using the `NumPy` library for numerical computations and `Matplotlib` for visualizing the robot motion and results. The inequality task definitions for joint limits and obstacle avoidance are described below.

2.1 Joint limits task

2.1.1 Task definition

The inequality task for joint limit avoidance is defined as

$$\sigma_{l_i} = \sigma_{l_i}(\mathbf{q}) = q_i, \tag{1}$$

where q_i denotes the i-th joint position.

The safe set for joint i is given by

$$\mathcal{S}_{l_i} = [q_{i,\min},\, q_{i,\max}], \tag{2}$$

and the task objective is to ensure

$$\sigma_{l_i} \in \mathcal{S}_{l_i}. \tag{3}$$

The corresponding task Jacobian is

$$\dot{x}_{l_i} = 1, \qquad \mathbf{J}_{l_i} = [0,\, 0,\, \ldots,\, 1,\, \ldots,\, 0] \in \mathbb{R}^{1\times n}, \tag{4}$$

where the non-zero element corresponds to the i-th joint.

2.1.2 Task Switching

Two thresholds are defined for task activation and deactivation:

$$\alpha_{l_i} \in \mathbb{R} \quad \text{(activation threshold)}, \tag{5}$$

$$\delta_{l_i} \in \mathbb{R} \quad \text{(deactivation threshold)}. \tag{6}$$

To avoid chattering, the thresholds satisfy

$$\delta_{l_i} > \alpha_{l_i}. \tag{7}$$

The activation function $a_{l_i}(\mathbf{q})$ is defined as

$$a_{l_i}(\mathbf{q}) = \begin{cases} -1, & a_{l_i} = 0 \ \wedge \ q_i \geq q_{i,\max} - \alpha_{l_i}, \\ 1, & a_{l_i} = 0 \ \wedge \ q_i \leq q_{i,\min} + \alpha_{l_i}, \\ 0, & a_{l_i} = -1 \ \wedge \ q_i \leq q_{i,\max} - \delta_{l_i}, \\ 0, & a_{l_i} = \ \ 1 \ \wedge \ q_i \geq q_{i,\min} + \delta_{l_i}. \end{cases} \tag{8}$$

Appendix A.1 shows the Python implementation of this task.

2.2 Obstacle avoidance task

2.2.1 Task Definition

The inequality task for end-effector obstacle avoidance is defined as

$$\sigma_r = \sigma_r(\mathbf{q}) = \|\eta_{1,ee}(\mathbf{q}) - \mathbf{P}\|\,, \tag{9}$$

where $\eta_{1,ee}(\mathbf{q}) \in \mathbb{R}^3$ denotes the end-effector position and $\mathbf{P}$ is the obstacle center.

The safe set is defined as

$$\mathcal{S}_r = \left\{ \eta_1 \in \mathbb{R}^3 \;\middle|\; \|\eta_1 - \mathbf{P}\| \leq r \right\}, \tag{10}$$

and the task objective is

$$\sigma_r \in \mathcal{S}_r. \tag{11}$$

The task Jacobian direction is given by

$$\dot{x}_r(\mathbf{q}) = \frac{\eta_{1,ee}(\mathbf{q}) - \mathbf{P}}{\|\eta_{1,ee}(\mathbf{q}) - \mathbf{P}\|}, \tag{12}$$

with the corresponding Jacobian

$$\mathbf{J}_r = \mathbf{J}_r(\mathbf{q}) = \mathbf{J}_v(\mathbf{q}) \in \mathbb{R}^{3\times n}. \tag{13}$$

2.2.2 Task Switching

Two thresholds are introduced:

$$r_\alpha \in \mathbb{R} \quad \text{(activation threshold)}, \tag{14}$$

$$r_\delta \in \mathbb{R} \quad \text{(deactivation threshold)}. \tag{15}$$

To avoid chattering, the thresholds satisfy

$$r_\delta > r_\alpha. \tag{16}$$

The activation function is defined as

$$a_r(\mathbf{q}) = \begin{cases} 1, & a_r = 0 \ \wedge \ \|\eta_{1,ee}(\mathbf{q}) - \mathbf{P}\| \leq r_\alpha, \\ 0, & a_r = 1 \ \wedge \ \|\eta_{1,ee}(\mathbf{q}) - \mathbf{P}\| \geq r_\delta. \end{cases} \tag{17}$$

The Python implementation of this task is shown in Appendix A.2.

3 Results

Two main experiments were conducted in this study. In the first experiment, an inequality task for obstacle avoidance was defined together with an end-effector position task. In the second experiment, a task-priority scheme incorporating an inequality task for joint limit avoidance was implemented alongside an end-effector position task.

In this laboratory experiment, a 3-DOF planar robot with three revolute joints is used. The robot model, including its coordinate frames and Denavit–Hartenberg (DH) parameters, is illustrated in Fig. 1.

The robot configuration is initialized according to the DH parameters listed in Table 1.

DOF	θ_i	d_i	a_i	α_i	**Home**
1	q_1	0	$a_1 = 0.5$	0	0.2
2	q_2	0	$a_2 = 0.5$	0	0.5
3	q_3	0	$a_3 = 0.5$	0	0.2

Table 1: Denavit–Hartenberg parameters of the 3-DOF robot arm

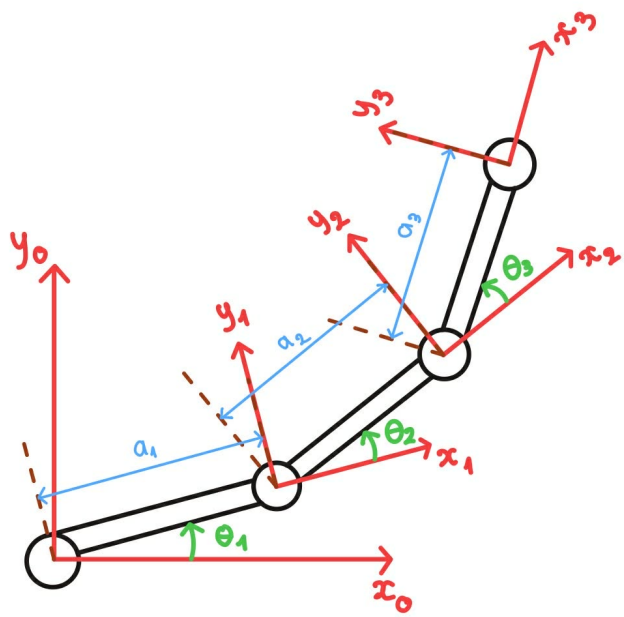


Figure 1: Visualization of the 3-DOF planar robot model

3.1 Obstacle avoidance task

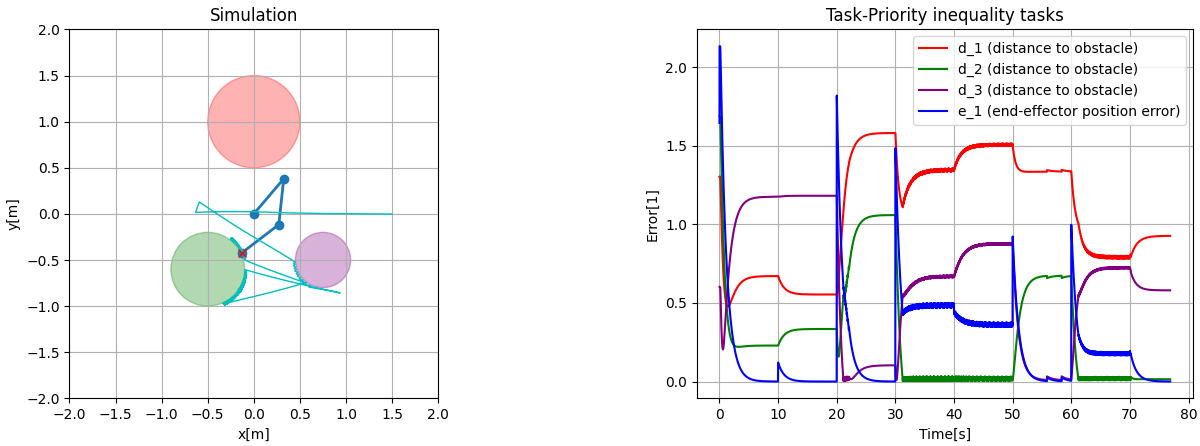
To conduct the first experiment, the total simulation time, sampling interval, and obstacle-related parameters were initialized. The parameters used in the simulation are summarized in Table 2.

Obstacle 1 position	$[0.0, 1.0]$	**Obstacle 3 position**	$[0.75, -0.5]$
Radius of obstacle 1	0.5	**Radius of obstacle 3**	0.3
Obstacle 2 position	$[-0.5, -0.6]$	r_α	0.01
Radius of obstacle 2	0.4	r_δ	0.02
Total simulation time	10	**Damping factor**	0.1

Table 2: Parameters for the obstacle avoidance experiments

After setting these parameters, the experiments were conducted, and the results are shown in Fig. 2a and Fig. 2b. The complete Python code implementation is provided in Appendices A.3 and A.4.

As shown in Fig. 2a, the robot end-effector successfully avoids the obstacles, and its entire trajectory remains outside the obstacle regions. However, a zig-zag behavior can be observed in the end-effector path when the robot enters the task-switching region for obstacle avoidance. This behavior is also reflected in Fig. 2b, where the distance to each obstacle does not reach zero but instead remains close to zero. This is because the obstacle avoidance task is activated at a distance of $r_\alpha = 0.01$ and deactivated at $r_\delta = 0.02$.



(a) Simulation of the manipulator, including end-effector goal and obstacles

(b) Evolution of the TP control error and distance to obstacles over time

Figure 2: Obstacle avoidance experiment results

Furthermore, in some cases, the end-effector does not reach the desired target position. This occurs either when the desired position lies inside an obstacle or when the available simulation time of 10 seconds is insufficient for the robot to reach the goal. Nevertheless, in all cases, the end-effector successfully avoids collisions with the obstacles.

3.2 Joint limit task

In this experiment, an inequality task for joint limit avoidance is defined together with a desired end-effector position task. The behavior of the robot arm is evaluated by analyzing the end-effector trajectory, the joint position evolution, and the end-effector position error.

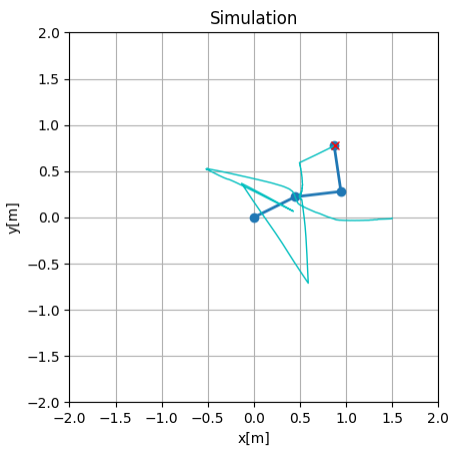
Before conducting the experiment, several parameters are defined and summarized in Table 3.

Joint 1 limit	$[-0.5, 0.5]$
Deactivation threshold (δ)	0.05
Activation thershold (α)	0.02
Total simulation time	10
Damping factor	0.1

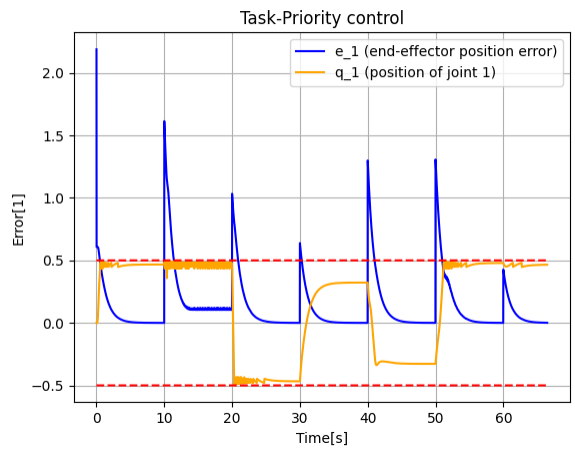
Table 3: Parameters for the joint limit experiment

As shown in Fig. 3a and Fig. 3b, the position of Joint 1 remains within the specified joint limits throughout the simulation. As the joint approaches its limits, the joint limit task is activated according to the defined activation and deactivation thresholds. Oscillatory behavior can be observed in both the end-effector trajectory and the joint position when the joint operates near its limits, which is a consequence of task switching.

Furthermore, as illustrated in Fig. 3b, the end-effector position error does not always converge to zero. This occurs when the desired end-effector target lies outside the reachable workspace due to the imposed joint limit constraints. This behavior confirms that the joint limit inequality task is successfully enforced, even at the expense of the primary task accuracy.



(a) Simulation of the manipulator, including the end-effector goal



(b) Evolution of the end-effector position error and joint position (including limits)

Figure 3: Joint limit experiment results

The complete python implementation is presented in the Appendices A.3 and A.5.

4 Conclusion

In conclusion, this laboratory exercise demonstrates the implementation of inequality tasks, including obstacle avoidance and joint limit constraints. Through simulation and analysis of the end-effector trajectory and joint positions, the resulting robot behaviors under different task configurations can be observed. When inequality tasks are enforced, the robot must satisfy these constraints while still attempting to accomplish the primary objective, such as reaching a desired end-effector position.

A Appendix: Code Implementations

This appendix contains all Python code implementations used in the laboratory exercise. To improve clarity and readability of the main report, all figures presenting source code are moved to this appendix.

A.1 Joint limit task

```python
'''
    Subclass of Task, representing the obstacle task.
'''
class JointLimit(Task):
    def __init__(self, name, safe_set, index):
        super().__init__(name, safe_set)
        self.err = 0.0 # Initialize with proper dimensions
        self.index = index
        self.ff_vel = np.zeros(1).reshape(1, 1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(1) # Initialize gain matrix K
        self.alpha = 0.02
        self.gamma = 0.05

    def update(self, robot):
        self.J = np.zeros((1, robot.getDOF())) # Update task Jacobian
        self.J[0, self.index] = 1 # Set the column corresponding to the joint index to 1
        self.err = np.array([[1]])
        joint_pos = robot.getJointPos(self.index)

        # Task switching logic
        if self.isActivate() == 0 and joint_pos >= (self.getDesired()[1] - self.alpha):
            self.a = -1
        elif self.isActivate() == 0 and joint_pos <= (self.getDesired()[0] + self.alpha):
            self.a = 1
        elif self.isActivate() == -1 and joint_pos <= (self.getDesired()[1] - self.gamma):
            self.a = 0
        elif self.isActivate() == 1 and joint_pos >= (self.getDesired()[0] + self.gamma):
            self.a = 0
```

A.2 Obstacle avoidance task

```
'''
    Subclass of Task, representing the obstacle task.
'''
class Obstacle2D(Task):
    def __init__(self, name, obstacle_pos, obs_boundary):
        super().__init__(name, obstacle_pos)
        self.err = 0.0 # Initialize with proper dimensions
        self.ff_vel = np.zeros(len(obstacle_pos)).reshape(2, 1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(len(obstacle_pos)) # Initialize gain matrix K
        self.radius_alpha = obs_boundary[0]
        self.radius_gamma = obs_boundary[1]

    def update(self, robot):
        self.J = robot.getLinkJacobian(3)[0:2, :] # Update task Jacobian
        dist_ee_obs = robot.getLinkTransform(3)[0:2, 3].reshape(2,1) - self.getDesired().reshape(2,1)
        self.err = dist_ee_obs/(abs(dist_ee_obs))

        # Task switching logic
        if self.isActivate() == 0 and np.linalg.norm(dist_ee_obs) <= self.radius_alpha:
            self.a = 1
        elif self.isActivate() == 1 and np.linalg.norm(dist_ee_obs) >= self.radius_gamma:
            self.a = 0
```

A.3 lab4_robotics.py

```
from lab2_robotics import * # Includes numpy import
from scipy.spatial.transform import Rotation as R

def jacobianLink(T, revolute, link): # Needed in Exercise 2
    '''
        Function builds a Jacobian for the end-effector of a robot,
        described by a list of kinematic transformations and a list of joint
            types.

        Arguments:
        T (list of Numpy array): list of transformations along the kinematic
            chain of the robot (from the base frame)
        revolute (list of Bool): list of flags specifying if the corresponding
            joint is a revolute joint
        link(integer): index of the link for which the Jacobian is computed

        Returns:
        (Numpy array): end-effector Jacobian
    '''
    # 1. Initialize J and O.
    J = np.zeros((6, link))
    O = T[-1][0:3, 3]
    # 2. For each joint of the robot
    for i in range(0, link):
        #   a. Extract z and o.
        i_1_T_i = T[i]
        z = i_1_T_i[0:3, 2]
        o = i_1_T_i[0:3, 3]
        #   b. Check joint type.
        if revolute[i] == True:
            cross_vector = np.array([np.cross(z, (O - o))]).T
            z = np.array([z]).T
            J_i = np.vstack((cross_vector, z))
        else:
            J_i = np.vstack((z, np.zeros((3, 1))))
        #   c. Modify corresponding column of J.
        J[:, i] = J_i[:, 0]
    return J


'''
    Class representing a robotic manipulator.
'''
class Manipulator:
    '''
        Constructor.

        Arguments:
        d (Numpy array): list of displacements along Z-axis
```

```
        theta (Numpy array): list of rotations around Z-axis
        a (Numpy array): list of displacements along X-axis
        alpha (Numpy array): list of rotations around X-axis
        revolute (list of Bool): list of flags specifying if the corresponding joint is a revolute joint
    '''
    def __init__(self, d, theta, a, alpha, revolute):
        self.d = d
        self.theta = theta
        self.a = a
        self.alpha = alpha
        self.revolute = revolute
        self.dof = len(self.revolute)
        self.q = np.zeros(self.dof).reshape(-1, 1)
        self.update(0.0, 0.0)

    '''
        Method that updates the state of the robot.

        Arguments:
        dq (Numpy array): a column vector of joint velocities
        dt (double): sampling time
    '''
    def update(self, dq, dt):
        self.q += dq * dt
        for i in range(len(self.revolute)):
            if self.revolute[i]:
                self.theta[i] = self.q[i][0]
            else:
                self.d[i] = self.q[i][0]
        self.T = kinematics(self.d, self.theta, self.a, self.alpha)

    '''
        Method that returns the characteristic points of the robot.
    '''
    def drawing(self):
        return robotPoints2D(self.T)

    '''
        Method that returns the end-effector Jacobian.
    '''
    def getEEJacobian(self):
        return jacobian(self.T, self.revolute)

    '''
        Method that returns the end-effector transformation.
    '''
    def getEETransform(self):
        return self.T[-1]

    '''
```

```
        Method that returns the transformation of a selected link.
    '''
    def getLinkTransform(self, link):
        return self.T[link]

    '''
        Method that returns the Jacobian of a selected link.
    '''
    def getLinkJacobian(self, link):
        return jacobianLink(self.T, self.revolute, link)

    '''
        Method that returns the position of a selected joint.

        Argument:
        joint (integer): index of the joint

        Returns:
        (double): position of the joint
    '''
    def getJointPos(self, joint):
        return self.q[joint]

    '''
        Method that returns number of DOF of the manipulator.
    '''
    def getDOF(self):
        return self.dof

'''
    Base class representing an abstract Task.
'''
class Task:
    '''
        Constructor.

        Arguments:
        name (string): title of the task
        desired (Numpy array): desired sigma (goal)
    '''
    def __init__(self, name, desired):
        self.name = name # task title
        self.sigma_d = desired # desired sigma
        self.a = 0 # Activation value
        self.ff_vel = None
        self.gain_matrix_K = None

    '''
        Method updating the task variables (abstract).

        Arguments:
```

```
        robot (object of class Manipulator): reference to the manipulator
    '''
    def update(self, robot):
        pass

    '''
        Method setting the desired sigma.

        Arguments:
        value(Numpy array): value of the desired sigma (goal)
    '''
    def setDesired(self, value):
        self.sigma_d = value

    '''
        Method returning the desired sigma.
    '''
    def getDesired(self):
        return self.sigma_d

    '''
        Method setting the feedforward velocity.

        Arguments:
        value(Numpy array): value of the feedforward velocity vector
    '''
    def setFeedforwardVelocity(self, value):
        self.ff_vel = value

    '''
        Method returning the feedforward velocity vector.
    '''
    def getFeedforwardVelocity(self):
        return self.ff_vel

    '''
        Method setting the gain matrix K.

        Arguments:
        value(Numpy array): value of the gain matrix K
    '''
    def setGainMatrixK(self, value):
        self.gain_matrix_K = value

    '''
        Method returning the gain matrix K.
    '''
    def getGainMatrixK(self):
        return self.gain_matrix_K

    '''
```

```
        Method returning the task Jacobian.
    '''
    def getJacobian(self):
        return self.J

    '''
        Method returning the task error (tilde sigma).
    '''
    def getError(self):
        return self.err

    '''
        Method checking the active state.
    '''
    def isActivate(self) -> bool:
        return self.a

'''
    Subclass of Task, representing the 2D position task.
'''
class Position2D(Task):
    def __init__(self, name, desired, link_index):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape) # Initialize with proper dimensions
        self.link_index = link_index # Index of the link for which the task is defined
        self.ff_vel = np.zeros(desired.shape) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(desired.shape[0]) # Initialize gain matrix K
        self.a = 1 # Initialize activate value of task

    def update(self, robot):
        self.J = robot.getLinkJacobian(self.link_index)[0:2,:]   # Update task Jacobian
        self.err = self.getDesired() - robot.getLinkTransform(self.link_index)[0:2, 3].reshape(2,1)# Update task error

'''
    Subclass of Task, representing the 2D orientation task.
'''
class Orientation2D(Task):
    def __init__(self, name, desired, link_index):
        super().__init__(name, desired)
        self.err = 0.0 # Initialize with proper dimensions
        self.link_index = link_index # Index of the link for which the task is defined
        self.ff_vel = np.zeros(1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(1) # Initialize gain matrix K
        self.a = 1 # Initialize activate value of task

```

```
    def update(self, robot):
        self.J = robot.getLinkJacobian(self.link_index)[5, :].reshape(1, self.link_index) # Update task Jacobian
        rotation_matrix_desired = R.from_euler('z', self.getDesired()).as_matrix() # Convert desired orientation from Euler angles to rotation matrix
        w_d, epsilon_d = self.quaternion_from_rotation_matrix(rotation_matrix_desired)
        w, epsilon = self.quaternion_from_rotation_matrix(robot.getLinkTransform(self.link_index)[0:3])
        self.err = (w * epsilon_d - w_d * epsilon - np.cross(epsilon, epsilon_d))[-1]# Update task error
        self.err = np.array(self.err).reshape(1, 1) # Reshape error to be a column vector

    # Method to convert rotation matrix to quaternion (scalar-first convention)
    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon
'''
    Subclass of Task, representing the 2D configuration task.
'''
class Configuration2D(Task):
    def __init__(self, name, desired, link_index):
        super().__init__(name, desired)
        self.err = np.zeros(desired.shape) # Initialize with proper dimensions
        self.link_index = link_index # Index of the link for which the task is defined
        self.ff_vel = np.zeros(desired.shape) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(desired.shape[0]) # Initialize gain matrix K
        self.a = 1 # Initialize activate value of task

    def update(self, robot):
        self.J = np.vstack((robot.getLinkJacobian(self.link_index)[0:2,:], robot.getLinkJacobian(self.link_index)[5, :])).reshape(3, self.link_index)# Update task Jacobian
        rotation_matrix_desired = R.from_euler('z', self.getDesired()[-1]).as_matrix() # Convert desired orientation from Euler angles to rotation matrix
        w_d, epsilon_d = self.quaternion_from_rotation_matrix(rotation_matrix_desired) # Desired orientation in quaternion form
        w, epsilon = self.quaternion_from_rotation_matrix(robot.getLinkTransform(self.link_index)[0:3]) # Current orientation in quaternion form
```

```
        self.err = np.vstack((self.getDesired()[0:2].reshape(2,1) - robot.getLinkTransform(self.link_index)[0:2, 3].reshape(2,1),
                              (w * epsilon_d - w_d * epsilon - np.cross(epsilon, epsilon_d))[-1]))# Update task error
        self.err = self.err.reshape(3, 1) # Reshape error to be a column vector

    def quaternion_from_rotation_matrix(self, orientation_matrix):
        orientation_matrix = R.from_matrix(orientation_matrix)
        quaternion = orientation_matrix.as_quat(scalar_first=True)
        w = quaternion[0]
        epsilon = quaternion[1:4]

        return w, epsilon

'''
    Subclass of Task, representing the joint position task.
'''
class JointPosition(Task):
    def __init__(self, name, index, desired):
        super().__init__(name, desired)
        self.err = 0.0 # Initialize with proper dimensions
        self.index = index # Index of the joint for which the task is defined
        self.ff_vel = np.zeros(1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(1) # Initialize gain matrix K
        self.a = 1 # Initialize activate value of task

    def update(self, robot):
        self.J = np.zeros((1, robot.getDOF())) # Update task Jacobian
        self.J[0, self.index] = 1 # Set the column corresponding to the joint index to 1
        self.err = self.getDesired() - robot.getJointPos(self.index) # Update task error
        self.err = np.array(self.err).reshape(1, 1) # Reshape error to be a column vector

'''
    Subclass of Task, representing the obstacle task.
'''
class Obstacle2D(Task):
    def __init__(self, name, obstacle_pos, obs_boundary):
        super().__init__(name, obstacle_pos)
        self.err = 0.0 # Initialize with proper dimensions
        self.ff_vel = np.zeros(len(obstacle_pos)).reshape(2, 1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(len(obstacle_pos)) # Initialize gain matrix K
        self.radius_alpha = obs_boundary[0]
        self.radius_gamma = obs_boundary[1]

    def update(self, robot):
```

```
        self.J = robot.getLinkJacobian(3)[0:2, :] # Update task Jacobian
        dist_ee_obs = robot.getLinkTransform(3)[0:2, 3].reshape(2,1) - self.getDesired().reshape(2,1)
        self.err = dist_ee_obs/(abs(dist_ee_obs))

        # Task switching logic
        if self.isActivate() == 0 and np.linalg.norm(dist_ee_obs) <= self.radius_alpha:
            self.a = 1
        elif self.isActivate() == 1 and np.linalg.norm(dist_ee_obs) >= self.radius_gamma:
            self.a = 0

'''
    Subclass of Task, representing the obstacle task.
'''
class JointLimit(Task):
    def __init__(self, name, safe_set, index):
        super().__init__(name, safe_set)
        self.err = 0.0 # Initialize with proper dimensions
        self.index = index
        self.ff_vel = np.zeros(1).reshape(1, 1) # Initialize feedforward velocity
        self.gain_matrix_K = np.eye(1) # Initialize gain matrix K
        self.alpha = 0.02
        self.gamma = 0.05

    def update(self, robot):
        self.J = np.zeros((1, robot.getDOF())) # Update task Jacobian
        self.J[0, self.index] = 1 # Set the column corresponding to the joint index to 1
        self.err = np.array([[1]])
        joint_pos = robot.getJointPos(self.index)

        # Task switching logic
        if self.isActivate() == 0 and joint_pos >= (self.getDesired()[1] - self.alpha):
            self.a = -1
        elif self.isActivate() == 0 and joint_pos <= (self.getDesired()[0] + self.alpha):
            self.a = 1
        elif self.isActivate() == -1 and joint_pos <= (self.getDesired()[1] - self.gamma):
            self.a = 0
        elif self.isActivate() == 1 and joint_pos >= (self.getDesired()[0] + self.gamma):
            self.a = 0
```

A.4 Full code for obstacle avoidance task

```
from lab4_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim
import matplotlib.patches as patch

# Robot model
d = np.zeros(3)                                    # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])                  # rotation around Z-axis
alpha = np.zeros(3)                                # rotation around X-axis
a = np.array([0.5, 0.5, 0.5])                      # displacement along X-axis
revolute = np.array([True, True, True])            # flags specifying the type of joints
robot = Manipulator(d, theta, a, alpha, revolute) # Manipulator object
damping_factor = 0.1

# Task hierarchy definition
obstacle_pos_1 = np.array([0.0, 1.0]).reshape(2,1)
obstacle_pos_2 = np.array([-0.5, -0.6]).reshape(2,1)
obstacle_pos_3 = np.array([0.75, -0.5]).reshape(2,1)
obstacle_r_1 = 0.5
obstacle_r_2 = 0.4
obstacle_r_3 = 0.3

tasks = [
          Obstacle2D("Obstacle avoidance", obstacle_pos_1, np.array([obstacle_r_1+0.01, obstacle_r_1+0.02])),
          Obstacle2D("Obstacle avoidance", obstacle_pos_2, np.array([obstacle_r_2+0.01, obstacle_r_2+0.02])),
          Obstacle2D("Obstacle avoidance", obstacle_pos_3, np.array([obstacle_r_3+0.01, obstacle_r_3+0.02])),
          Position2D("End-effector position", np.array([-1.0, 1.0]).reshape(2,1), 3)
        ]

# Simulation params
dt = 1.0/60.0

# Record the data for plotting
error_end_effector = []
error_obstacle_1 = []
error_obstacle_2 = []
error_obstacle_3 = []

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
```

```
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
ax.add_patch(patch.Circle(obstacle_pos_1.flatten(), obstacle_r_1, color='red',
    alpha=0.3))
ax.add_patch(patch.Circle(obstacle_pos_2.flatten(), obstacle_r_2,
    color='green', alpha=0.3))
ax.add_patch(patch.Circle(obstacle_pos_3.flatten(), obstacle_r_3,
    color='purple', alpha=0.3))
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target

PPx = []
PPy = []

# Simulation initialization
def init():
    global tasks
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    tasks[3].setDesired((np.random.rand(2,1)*2-1)) # Random target
    return line, path, point

# Simulation loop
def simulate(t):
    global tasks
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm (w/set-based tasks)
    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):

        tasks[i].update(robot)

        if tasks[i].isActivate() != 0:
            # Update task state
            # Padding Jacobian to match the dimension of the robot dof and
                compute the augmented Jacobian, then compute the DLS solution
                and accumulate velocity, and finally update the null-space
                projector
            Jacobian_padded = np.pad(tasks[i].getJacobian(), ((0, 0), (0,
                robot.dof - tasks[i].getJacobian().shape[1])))
            # Compute augmented Jacobian
            J_bar = Jacobian_padded @ P
```

```
            # Task definition
            x_dot = tasks[i].getFeedforwardVelocity() + tasks[i].getGainMatrixK() @ tasks[i].getError()
            # Compute task velocity & Accumulate velocity
            dq = dq + DLS(J_bar, damping_factor) @ (tasks[i].a * x_dot - Jacobian_padded @ dq)
            # Update null-space projector
            P = P - np.linalg.pinv(J_bar) @ J_bar
        else:
            dq = dq
            P = P
    ###

    # Record data for plotting
    error_end_effector.append(np.linalg.norm(tasks[3].getError()))
    error_obstacle_1.append(np.linalg.norm(robot.getEETransform()[0:2,3] - obstacle_pos_1.flatten()) - obstacle_r_1)
    error_obstacle_2.append(np.linalg.norm(robot.getEETransform()[0:2,3] - obstacle_pos_2.flatten()) - obstacle_r_2)
    error_obstacle_3.append(np.linalg.norm(robot.getEETransform()[0:2,3] - obstacle_pos_3.flatten()) - obstacle_r_3)

    # Update robot
    robot.update(dq, dt)

    # Update drawing
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(tasks[3].getDesired()[0], tasks[3].getDesired()[1])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                               interval=10, blit=True, init_func=init, repeat=True)
plt.show()

# Plotting the end-effector path and distance to the obstacles
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
plt.plot(tvec, error_obstacle_1, label='d_1 (distance to obstacle)', color='red')
plt.plot(tvec, error_obstacle_2, label='d_2 (distance to obstacle)', color='green')
plt.plot(tvec, error_obstacle_3, label='d_3 (distance to obstacle)', color='purple')
```

```
plt.plot(tvec, error_end_effector, label='e_1 (end-effector position error)',
    color='blue')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.title('Task-Priority inequality tasks')
plt.legend()
plt.grid()
plt.show()
```

A.5 Full code for joint limits task

```
from lab4_robotics import * # Includes numpy import
import matplotlib.pyplot as plt
import matplotlib.animation as anim
import matplotlib.patches as patch

# Robot model
d = np.zeros(3)                                  # displacement along Z-axis
theta = np.array([0.2, 0.5, 0.2])                # rotation around Z-axis
alpha = np.zeros(3)                              # rotation around X-axis
a = np.array([0.5, 0.5, 0.5])                    # displacement along X-axis
revolute = np.array([True, True, True])          # flags specifying the type of joints
robot = Manipulator(d, theta, a, alpha, revolute) # Manipulator object
damping_factor = 0.1

# Task hierarchy definition
joint_limit_1 = np.array([-0.5, 0.5])

tasks = [
          JointLimit("Joint Limit", joint_limit_1, 0),
          Position2D("End-effector position", np.array([-1.0, 1.0]).reshape(2,1), 3)
        ]

# Simulation params
dt = 1.0/60.0

# Record the data for plotting
error_end_effector = []
joint_1_position = []

# Drawing preparation
fig = plt.figure()
ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2,2))
ax.set_title('Simulation')
ax.set_aspect('equal')
ax.grid()
ax.set_xlabel('x[m]')
ax.set_ylabel('y[m]')
line, = ax.plot([], [], 'o-', lw=2) # Robot structure
path, = ax.plot([], [], 'c-', lw=1) # End-effector path
point, = ax.plot([], [], 'rx') # Target

PPx = []
PPy = []

# Simulation initialization
def init():
    global tasks
```

```
    line.set_data([], [])
    path.set_data([], [])
    point.set_data([], [])
    tasks[1].setDesired((np.random.rand(2,1)*2-1)) # Random target
    return line, path, point

# Simulation loop
def simulate(t):
    global tasks
    global robot
    global PPx, PPy

    ### Recursive Task-Priority algorithm (w/set-based tasks)
    ### Recursive Task-Priority algorithm
    # Initialize null-space
    P = np.identity(robot.dof)
    # Initialize output vector (joint velocity)
    dq = np.zeros((robot.dof, 1))
    # Loop over tasks
    for i in range(len(tasks)):

        tasks[i].update(robot)

        if tasks[i].isActivate() != 0:
            # Update task state
            # Padding Jacobian to match the dimension of the robot dof and compute the augmented Jacobian, then compute the DLS solution and accumulate velocity, and finally update the null-space projector
            Jacobian_padded = np.pad(tasks[i].getJacobian(), ((0, 0), (0, robot.dof - tasks[i].getJacobian().shape[1])))
            # Compute augmented Jacobian
            J_bar = Jacobian_padded @ P
            # Task definition
            x_dot = tasks[i].getFeedforwardVelocity() + tasks[i].getGainMatrixK() @ tasks[i].getError()
            # Compute task velocity & Accumulate velocity
            dq = dq + DLS(J_bar, damping_factor) @ (tasks[i].a * x_dot - Jacobian_padded @ dq)
            # Update null-space projector
            P = P - np.linalg.pinv(J_bar) @ J_bar
        else:
            dq = dq
            P = P
    ###

    # Record data for plotting
    error_end_effector.append(np.linalg.norm(tasks[1].getError()))
    joint_1_position.append(robot.getJointPos(0)[0])

    # Update robot
```

```
    robot.update(dq, dt)

    # Update drawing
    PP = robot.drawing()
    line.set_data(PP[0,:], PP[1,:])
    PPx.append(PP[0,-1])
    PPy.append(PP[1,-1])
    path.set_data(PPx, PPy)
    point.set_data(tasks[1].getDesired()[0], tasks[1].getDesired()[1])

    return line, path, point

# Run simulation
animation = anim.FuncAnimation(fig, simulate, np.arange(0, 10, dt),
                                interval=10, blit=True, init_func=init,
                                   repeat=True)
plt.show()

# Plotting the end-effector path and distance to the obstacles
tvec = np.arange(0, len(error_end_effector)*dt, dt)
plt.figure()
plt.plot(tvec, error_end_effector, label='e_1 (end-effector position error)',
    color='blue')
plt.plot(tvec, joint_1_position, label='q_1 (position of joint 1)',
    color='orange')
plt.plot(tvec, np.ones_like(tvec)*joint_limit_1[0], color='red',
    linestyle='--')
plt.plot(tvec, np.ones_like(tvec)*joint_limit_1[1], color='red',
    linestyle='--')
plt.xlabel('Time[s]')
plt.ylabel('Error[1]')
plt.title('Task-Priority control')
plt.legend()
plt.grid()
plt.show()
```